

Reto Hackathon: Arquitectura Basada en Eventos e Integración con LLMs

Objetivo:

Diseñar e implementar una solución innovadora basada en Inteligencia Artificial (LLMs) que resuelva un problema real a elección del equipo. El enfoque principal no es sólo la invocación del modelo, sino el diseño de una arquitectura basada en eventos, asíncrona y fácilmente desplegable. Además, los equipos deberán proporcionar un contexto claro sobre la problemática abordada y los casos de uso propuestos.

Lineamientos Técnicos Obligatorios:

- **Arquitectura Basada en Eventos:**

La solución debe incluir un flujo de trabajo asíncrono y orientado a eventos. No se limita el uso a una sola nube; los estudiantes pueden elegir el proveedor con el que estén más familiarizados (AWS, GCP, Azure, OCI, etc.), utilizando servicios como AWS SQS, AWS EventBridge, Google Cloud Pub/Sub, entre otros. Se debe tratar en lo posible que toda la arquitectura sea serverless. Excepcionalmente, se permitirá el uso de una Máquina Virtual (MV) con Docker para levantar componentes específicos que lo requieran estrictamente (por ejemplo, una base de datos vectorial para un enfoque RAG).

- **Interfaz de Usuario (Frontend):**

Se debe desarrollar un frontend desde donde se pueda lanzar la ejecución del flujo de trabajo y visualizar los resultados. Este frontend podrá ser desplegado en cualquiera de las nubes mencionadas anteriormente. Además, debe permitir eventualmente la subida de un archivo o archivos con los inputs que se van a procesar de forma masiva.

- **Resiliencia y Manejo de Límites de API:**

El sistema debe integrarse con una API de LLM (como Groq - <https://console.groq.com/keys>). Se debe demostrar el procesamiento masivo pero controlado de información (lotes pequeños de 20 a 30 elementos). Es necesario implementar mecanismos de control de errores y políticas de reintentos para manejar posibles bloqueos o restricciones por límites de peticiones de la API.

- **Despliegue Reproducible:**

Para usar la plataforma, los equipos deben incluir una documentación con un manual paso a paso detallado para que se pueda probar la solución sin inconvenientes. Si utilizaron componentes Docker de forma excepcional, deben proveer un docker-compose bien estructurado.

Entregables Esperados

- **Repositorio de Código:** El código fuente completo de la solución (frontend, backend y funciones serverless).
- **Contexto de la Problemática:** Un documento o sección en el repositorio que explique el problema real que se busca resolver, los casos de uso y el impacto esperado.
- **Manual de Despliegue:** Documentación paso a paso para levantar y probar la plataforma, junto con el archivo docker-compose (en caso de aplicar).
- **Diagrama de Arquitectura:** Un Diagrama de Arquitectura de Solución que explique el flujo de eventos, los datos y los servicios de nube utilizados.

- **Enlace del Frontend Desplegado:** La URL pública funcional para acceder y probar la interfaz de usuario directamente en la nube.
- **Video en youtube** con una demo completa de las funcionalidades.

Ejemplo de Referencia: (Disclaimer: Este caso de uso es solo explicativo para entender la dinámica esperada y **NO** puede ser presentado por los equipos en la hackathon).

Caso de uso referencial: Clasificador automático de feedback de clientes.

Paso a paso del flujo:

- Desde el frontend, el usuario sube un archivo CSV con 30 comentarios de clientes.
- El backend lee el archivo y dispara un evento independiente por cada comentario hacia una cola de mensajes o bus de eventos (ej. AWS SQS, GCP Pub/Sub, o AWS EventBridge).
- Una función serverless consume estos eventos de la cola de forma asíncrona y envía el texto a la API del LLM (Groq) para clasificar el sentimiento (positivo, negativo, neutral).
- Si la API del LLM rechaza una petición porque se alcanzó el límite de uso masivo, la función captura el error y devuelve el mensaje a la cola con un retraso (reintento) para procesarlo más tarde sin perder la data.
- Finalmente, los resultados procesados se guardan y el usuario puede visualizar en el frontend un panel con la clasificación final de cada uno de los comentarios.

Rúbrica de Evaluación del Hackathon:

Criterio de Evaluación	Excelente (4 pts)	Bueno (3 pts)	Regular (2 pts)	Deficiente (1 - 0 pts)
1. Contexto de la Problemática e Impacto <i>(Entregable: Documento/Sección de Contexto)</i>	Define de manera clara y profunda un problema real. Los casos de uso son altamente relevantes y el impacto esperado de la solución con LLM está perfectamente justificado.	Define un problema real y casos de uso aceptables, pero la justificación del impacto o la conexión con la solución de IA carece de un poco de profundidad.	El problema y los casos de uso se mencionan de forma superficial o genérica. El impacto esperado no está claro.	No se incluye el documento de contexto o no se logra entender qué problema real intenta resolver la solución.

2. Diseño y Diagrama de Arquitectura <i>(Entregable: Diagrama de Arquitectura)</i>	Presenta un diagrama gráfico claro y preciso. La arquitectura es 100% basada en eventos, asíncrona y predominantemente <i>serverless</i> . (El uso de Docker/MV, si lo hay, está estrictamente justificado).	Presenta un diagrama comprensible. La arquitectura es mayormente asíncrona y <i>serverless</i> , pero el flujo de eventos tiene cuellos de botella menores o dependencias síncronas innecesarias.	El diagrama es confuso o incompleto. La solución depende fuertemente de procesos síncronos o el uso de máquinas virtuales no está justificado (ej. monolito).	No hay diagrama o la arquitectura propuesta no utiliza servicios de nube ni un enfoque basado en eventos.
3. Resiliencia, Manejo de Límites y LLM <i>(Lineamiento: Código Backend/Cola de Eventos)</i>	Implementa a la perfección el procesamiento masivo por lotes (20-30). Captura errores de la API del LLM eficientemente y ejecuta políticas de reintento automático mediante colas (ej. SQS/PubSub) sin perder datos.	Procesa la información en lotes y tiene manejo de errores básico. Cuenta con políticas de reintento, pero su implementación podría fallar bajo cargas extremas o es ligeramente ineficiente.	Intenta procesar múltiples elementos pero de forma secuencial o síncrona. El manejo de errores es frágil y carece de una política de reintentos mediante encolamiento.	No hay integración real con un LLM, no soporta procesamiento masivo de datos, o la aplicación colapsa al primer límite de API.

4. Frontend y Experiencia de Usuario <i>(Entregable: Enlace del Frontend Desplegado)</i>	La URL pública es completamente funcional. Permite subir archivos fácilmente, dispara el flujo de trabajo de forma exitosa y visualiza los resultados procesados de manera clara e intuitiva.	El enlace es público y funcional. Permite subir el input y ver resultados, pero la interfaz carece de pulido o la visualización de datos es demasiado básica.	El frontend está desplegado pero presenta fallos al intentar subir archivos, o no logra mostrar los resultados devueltos por el backend asíncrono.	No se entregó una URL pública, el enlace está roto, o no se desarrolló ninguna interfaz de usuario.
5. Repositorio, Código y Despliegue <i>(Entregables: Repositorio y Manual)</i>	Código limpio y completo (Front, Back, Serverless). El manual paso a paso es impecable y garantiza un despliegue sin fricciones. Incluye un docker-compose perfectamente estructurado (si aplica).	Código completo y documentado . El manual es bueno pero omite detalles menores que requieren que el evaluador deduzca algún paso para lograr el despliegue exitoso.	Faltan secciones del código. El manual es vago, incompleto o los comandos fallan, dificultando severament e probar la plataforma de manera local o en la nube.	Repositorio inaccesible o incompleto. No hay manual de despliegue, haciendo imposible reproducir el proyecto.